

Mobile Application Development Lab

Dimitrios Ringas

riggas@ionio.gr

Department of Informatics

Ionian University



Lab 1 – JavaScript Fundamentals

Goal: Get familiar with modern JavaScript (ES6+) syntax and logic – everything we will need for React Native.



JavaScript - origins

JavaScript was created in 1995 by Netscape, as a way to add executable code to web pages and make them *alive*.

It was originally called LiveScript, but due to the popularity of Java at the time it was renamed to JavaScript.

However, it has nothing else to do with Java

As it spread to other browsers (*and after becoming a major battleground during the browser wars*) it was standardized as **ECMAScript**.

- ✓ Today the terms **JavaScript** and **ECMAScript** are often used interchangeably.
- ✓ Since 2015 (**ES6** / ES2015) a new version is released **every year** — "modern JavaScript" = ES6+

JavaScript - characteristics

A high-level programming language, **dynamically typed**

An interpreted language — in practice, modern engines perform **JIT compilation**

JavaScript does not run only inside browsers.

✓ it runs in any environment that includes a compatible **JavaScript engine**, e.g.

✓ V8 (Google), used in:

✓ Chrome, Opera, Edge, etc.

✓ Node.js, Deno, etc.

✓ SpiderMonkey (Mozilla)

✓ Firefox

✓ Netscape (in the past)

✓ JavaScriptCore (Apple)

✓ Safari

✓ WebKit browsers

✓ **Hermes** (Meta)

✓ React Native 

✓ It is **event-driven** and **single-threaded**

✓ Modern JS uses **modules**, **arrow functions** and **promises**

.

(a short pause)

what do we remember from Web Technologies?

(let's just look at a few interesting bits here)



Variables: `const`, `let` (and `var`)

`const` – cannot be reassigned

```
const name = "Eleni";  
name = "Maria";    // ✗ TypeError  
  
const user = { name: "Eleni" };  
user.name = "Maria"; // ✓ the object  
                  // changes, the reference doesn't
```

`let` – value can change

```
let count = 0;  
count = count + 1;    // ✓
```

`var` – the old way, **avoid it**
(function scope, not block scope)

Rule: `const` everywhere; `let` only when the value **must** change; never `var`.
Never assign without declaring (`x = 5`) – in modules it is a `ReferenceError`.



Types, equality & truthy / falsy

```
typeof 42           // 'number'
typeof "hi"         // 'string'
typeof true         // 'boolean'
typeof undefined    // 'undefined'
typeof null         // 'object' (historical bug!)
typeof [1, 2]       // 'object'
typeof {}           // 'object'
typeof (() => 1)     // 'function'
Array.isArray([1])  // true
```

Always **===** (no type coercion)

```
0 == ""           // true 🤖
0 === ""          // false ✅
"5" == 5          // true 🤖
"5" === 5         // false ✅
```

Falsy: **false**, **0**, **""**, **null**, **undefined**, **NaN**

Everything else is **truthy** (including **[]**, **{}**!)

Functions

- A function is a block of code that can be called.
 - It does not declare a return type, or whether it returns anything
 - It may return something, or not (in which case it returns **undefined**)
 - It may take parameters, specifying only their names, not their types
 - It may have a name, but it can also be anonymous
 - It is a **value**: it can be passed as an argument to another function (callback)

```
function hello(){    // Definition
    console.log("Hello");
}
hello();             // Call
```

```
function greetingStr(name){
    return "Hello "+name;
}
console.log( greetingStr("Eleni") );
```

```
const f = function(){
    console.log("This is anonymous");
};
f(); // prints: This is anonymous
const g = f;
g(); // prints: This is anonymous
```

```
const f = function(){
    console.log("This is anonymous");
}; // ⚠ without ; the next ( calls f!
(function(fun) {
    console.log("Calling another function");
    fun();
})(f); // Defining and calling with a param
```

Functions & scope

- Variables and constants declared with **let** and **const** are local to the block where they are defined, but be careful..
- whatever is defined in a block is local there
- a function looks up in outer blocks

```
let x=10;
for(let i = 0; i < 10; i++){
  x += i;
}
console.log(x); // prints 55
typeof i;       // returns 'undefined'
```

```
let x = 10;
function increase(){
  x +=1 ;
}
increase();
console.log(x); // prints 11
```

- variables with the same name in an inner block shadow those in outer ones

```
let x = 10;
function multiply(z){
  let x = 5;
  return x * z;
}
multiply(x); // returns 50, not 100
typeof z;    // returns 'undefined'
```

- function** declarations are "lifted" to the top (**hoisting**) – **const f = () => ...** are not

```
isInScope(); // ✓
function isInScope(){
  console.log("Defined below its call ?!");
}
later();      // ✗ ReferenceError
const later = () => console.log("no");
```

Arrow functions

- Functions can also be defined with a shorter syntax:

classic syntax:

```
function hello(){  
    console.log("Hello");  
}
```

multiline arrow function:

```
const hello = () => {  
    console.log("Hello");  
};
```

single **expression** arrow function:

```
const hello = () => console.log("Hello");
```

arrow function with a parameter:

```
const hello = (name) => console.log("Hello "+name);
```

or, *for a single parameter*, even simpler:

```
const hello = name => console.log("Hello "+name);
```

arrow function returning a value (without **return**):

```
const hello = () => "Hello World!";
```

returning an **object**: wrap it in parentheses!

```
const mk = (id) => ({ id, done: false });
```

Function parameters

- If a function is called with more arguments than it declares, the extra ones are ignored without any error.
- A function can declare some parameters as optional, giving each one the value to use if it is not provided in the call
 - or the function body can check whether a parameter was provided and run the corresponding code

```
const power = (num, exp=2) => num ** exp ;  
power(3);      // returns 9  
power(2, 10)  // returns 1024
```

```
function minus(a, b) {  
  if (b === undefined) return -a;  
  return a - b;  
}  
minus(10);      // returns -10  
minus(10, 5);   // returns 5
```

Template Strings (or Template Literals)

They are delimited with backtick characters ```, allowing multi-line strings, string interpolation with embedded expressions, and special constructs called tagged templates.

Some examples:

```
`string text`  
  
`string text line 1  
string text line 2`  
  
`string text ${expression} string text`  
  
tagFunction`string text ${expression} string text`
```

Template Strings (or Template Literals)

Printing a multi-line string

```
console.log(`string text line 1  
string text line 2`);
```

```
string text line 1  
string text line 2
```

Embedding expressions (no concatenation)

```
const a = 5;  
const b = 10;  
console.log(`Fifteen is ${a + b} and not ${2 * a + b}.`);
```

```
Fifteen is 15 and not 20.
```

Tagged templates

```
const person = "Mike";  
const age = 28;  
function myTag(strings, personExp, ageExp) {  
  const str0 = strings[0]; // "That "  
  const str1 = strings[1]; // " is a "  
  const str2 = strings[2]; // "."  
  const ageStr = ageExp < 100 ? "youngster" : "centenarian";  
  // We can even return a string built using a template literal  
  return `${str0}${personExp}${str1}${ageStr}${str2}`;  
}  
const output = myTag`That ${person} is a ${age}.`;  
console.log(output);
```

The first argument of the tag function contains an array of string values.

The remaining arguments correspond to the expressions/parameters.

```
That Mike is a youngster.
```

Destructuring

The destructuring syntax makes it possible to unpack values from arrays, or properties from objects, into distinct variables.

Objects (by property name)

```
const user = { id: 7, username: "mark" };
const { id, username } = user;
console.log(`${username}'s ID is ${id}`)
// mark's ID is 7

// renaming & default value
const { username: nick, role = "student" } = user;
```

In function parameters

```
function Card({ title, year }) { /* ... */ }
Card({ title: "Dune", year: 1965 });
```

Arrays (by position)

```
const colors = ["red", "green", "blue"];
const [first, second] = colors;
// first = "red", second = "green"

const [, , third] = colors; // "blue"
```

📱 You will see this **all the time** in React:

```
const [count, setCount] = useState(0);
```

Spread & Rest: ...

Spread — "spread out" the elements

```
const a = [1, 2];
const b = [...a, 3];           // [1, 2, 3]
const c = [0, ...a];          // [0, 1, 2]

const user = { name: "Eleni", age: 20 };
const older = { ...user, age: 21 };
// { name: "Eleni", age: 21 }
// user has NOT changed
```

Rest — "gather" the rest

```
const sum = (...nums) =>
  nums.reduce((acc, n) => acc + n, 0);
sum(1, 2, 3);                  // 6

const [head, ...tail] = [1, 2, 3];
// head = 1, tail = [2, 3]

const { age, ...rest } = user;
// rest = { name: "Eleni" }
```

■ This is how we update state in React: a **new** copy, not a change to the old one

Arrays: `map`, `filter`, `find`, `reduce`

They return a **new** value – they don't change the original array

```
const courses = [
  { id: 1, title: "Mobile Apps", ects: 6, done: false },
  { id: 2, title: "Web Tech", ects: 5, done: true },
  { id: 3, title: "Databases", ects: 6, done: true },
];

courses.map(c => c.title);           // ["Mobile Apps", "Web Tech", "Databases"]
courses.filter(c => c.done);         // the 2 completed ones
courses.find(c => c.id === 2);       // { id: 2, title: "Web Tech", ... }
courses.reduce((sum, c) => sum + c.ects, 0); // 17
courses.some(c => !c.done);          // true

// Chaining: titles of completed courses, alphabetically
courses.filter(c => c.done).map(c => c.title).toSorted();
```

⚠ `push`, `pop`, `splice`, `sort` **mutate** the array · `toSorted()` returns a new one (ES2023)

📦 `.map()` is how we render lists in React

Values vs References

Primitives are copied

```
let a = 5;  
let b = a;  
b = 10;  
console.log(a); // 5
```

Objects / arrays: the reference is copied

```
const list = [1, 2];  
const same = list;  
same.push(3);  
console.log(list); // [1, 2, 3] 🙌
```

■ React detects that state has changed **only** if it receives a **new** reference

Comparison = "is it the **same** object?"

```
[1, 2] === [1, 2] // false  
const x = [1, 2];  
x === x // true
```

Copy (shallow) with spread

```
const copy = [...list];  
copy.push(4); // list does not change
```

Objects: modern syntax

Shorthand properties

```
const name = "Eleni", age = 20;  
const user = { name, age };  
// instead of { name: name, age: age }
```

Ternary & && (short-circuit evaluation)

```
const label = age >= 18 ? "adult" : "minor";  
isAdmin && console.log("Welcome admin");
```

Optional chaining **?.**

```
const u = { address: null };  
u.address.city;      // ✗ TypeError  
u.address?.city;     // undefined ✓
```

Nullish coalescing **??**

```
const n = 0;  
n || 10;    // 10 (0 is falsy)  
n ?? 10;    // 0 (only for null/undefined)
```

- Useful with data from APIs, where some fields may be missing

Asynchronous JavaScript

JavaScript uses an **event loop**.

Asynchronous mechanisms are **callbacks**, **promises** and **async/await**.

JavaScript runs on a single thread, which means it can only do one thing at a time. However, it manages to handle multiple tasks concurrently, thanks to the event loop.

- Call stack
Keeps track of which function is currently running.
- Web APIs / Background tasks
Functions such as `setTimeout`, `fetch` or event listeners are handed off to the runtime environment (browser or Node.js), and the next statement continues immediately.
- Queues (Microtask queue for Promises, Task / Callback queue for timers & events)
Once these background tasks complete, their callbacks are placed in a queue.
- Event loop
The event loop constantly checks *Is the call stack empty? If yes, it takes the first callback from the queue and pushes it onto the stack to be executed.*

Asynchronous JavaScript - callbacks

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Timeout");  
}, 0);  
  
console.log("End");
```

```
Start  
End  
Timeout
```

Even though `setTimeout` is set to 0, it still goes through the event loop and runs after the synchronous code.



Asynchronous JavaScript - microtasks vs tasks

```
console.log("A");

setTimeout(() => console.log("timeout"), 0);

Promise.resolve()
  .then(() => console.log("promise"));

console.log("B");
```

A
B
promise
timeout

- First, **all** the synchronous code runs
- Then **all** the microtasks (Promises, **await**)
- Then the next task (timers, events)

Asynchronous JavaScript - promises

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

```
[
  {
    id: 1,
    name: 'Leanne Graham',
    username: 'Bret',
    email: 'Sincere@april.biz',
    address: {
      street: 'Kulas Light',
      suite: 'Apt. 556',
      city: 'Gwenborough',
      zipcode: '92998-3874',
      geo: [Object]
    },
  },
  ...
]
```

- ✓ `fetch()` returns a Promise that resolves to a Response object once the request completes.
- ✓ `.then()` handles the Response object resolved by the `fetch()` call.
- ✓ `res.json()` is a method that reads the response stream and parses it as JSON.
It also returns a Promise, which resolves to the actual data (in this case, an array of user objects).
- ✓ It receives the parsed JSON data and `console.log(data)` prints the data to the console.

Asynchronous JavaScript - async/await

async/await are syntax elements used wherever Promises are involved; they make asynchronous code easier to read and write, as if it were synchronous code.

Same functionality as before:

```
async function getUsers() {  
  try {  
    const res = await fetch("https://jsonplaceholder.typicode.com/users");  
    if (!res.ok) throw new Error(`HTTP ${res.status}`);  
    const users = await res.json();  
    console.log(users);  
  } catch (err) {  
    console.error(err);  
  }  
}  
getUsers();
```

easier-to-read syntax

- **await** only inside an **async** function (or at the top level of an ES module)
- An **async** function **always** returns a Promise
- **fetch** does **not** fail on 404/500 – we check **res.ok**

Closures

A function **remembers** the variables of the scope where it was created, even after that scope has finished

```
function makeCounter() {  
  let count = 0;           // "private"  
  return () => {  
    count += 1;  
    return count;  
  };  
}  
  
const next = makeCounter();  
next();    // 1  
next();    // 2  
const other = makeCounter();  
other();   // 1 (its own count)
```

Why do we care?

- Every callback (`onPress`, `setTimeout`, `.then`) is a closure
- It "sees" the values from the moment it was created

```
let msg = "old";  
const show = () => console.log(msg);  
msg = "new";  
show();    // "new" - it sees the variable,  
           // not a copy of its value
```

Using modules

- Modules let us split code into reusable files using export and import.
- Modules help organize and encapsulate logic – allowing cleaner, maintainable and testable pieces of code.

math.mjs

```
export const add = (a,b) => a+b;           // named
export default function mul(a,b){ return a*b; } // default
```

main.mjs

```
import mul, { add } from "./math.mjs";
console.log(add(2,3), mul(2,3)); // 5 6
```

Running with Node.js:

```
node main.mjs
```

- **.mjs** → ES module
- or **"type": "module"** in **package.json** and **.js** files
- One **default** per file, as many **named** as we like
-  **export default function App()** in React Native

From JavaScript to React Native

What we saw today, as you will encounter it in the upcoming labs:

JavaScript	React Native
Array destructuring	<code>const [count, setCount] = useState(0);</code>
Object destructuring in parameters	<code>function Card({ title, year }) { ... } (props)</code>
Arrow functions & callbacks	<code>onPress={() => setCount(c => c + 1)}</code>
Spread / immutability	<code>setItems(prev => [...prev, newItem])</code>
<code>.map()</code>	<code>{items.map(i => <Text key={i.id}>{i.title}</Text>)}</code>
Ternary / <code>&&</code>	<code>{isLoading ? <Spinner /> : <List />}</code>
Template strings	<code><Text>{`Hello, \${name}!`}</Text></code>
<code>import</code> / <code>export default</code>	<code>import { View } from 'react-native';</code>
async / await	Loading data from APIs

Installation & setup

1. Install **Node.js** LTS (v22.13 +, v24 recommended) → <https://nodejs.org/en/download>
 - *We will use the same one for Expo/React Native from Lab 2 onwards*
2. Verify the installation

```
node -v  
npm -v
```

3. Create a working folder

```
mkdir lab1 && cd lab1  
code .
```

4. Each exercise in its own file, e.g. `ex1.js`, run with `node ex1.js`
 - For `import` / `export` or top-level `await`: `.mjs` files



Exercise 1 – Variables and types

Write a script that:

1. Declares 3 variables (`name` , `age` , `student`) – with `const` or `let` ? Why?
2. Prints them using template strings
3. Uses `typeof` to show each type
4. Compares `age == "20"` and `age === "20"` – explain the difference



Exercise 2 – Arrays & Loops

Create an array with your 5 favorite songs.

Use:

- a **for...of** loop to print each one
- **.map()** to create an array with the titles in uppercase [JavaScript Reference: .map\(\)](#).
- **.filter()** to keep only the titles longer than 10 characters
- Make sure the original array has **not** changed



Exercise 3 – Functions

Write a function `grade(score)` that:

- returns `"Pass"` if the score is ≥ 50
 - returns `"Fail"` otherwise
- Test it with various values.

Extension: Rewrite it as a one-line arrow function (with a ternary), and apply it to an array of scores with `.map()`.

💡 Exercise 4 – Objects & Destructuring

Define a **book** object with **title**, **author**, **year**.
Use destructuring to print:

“  title by author (year)”

Extension:

- Write a function **describe({ title, author })** that destructures in its parameters
- Create **updated** with a new **year** using spread – **book** must stay unchanged



Exercise 5 – Async Fetch

Write a function that fetches users from `https://jsonplaceholder.typicode.com/users` and prints only their names.

- Once with `.then()` and once with `async/await`
- What happens if you change the URL to `.../userz`? Handle the error.



Exercise 6 – Immutable list updates

Start from:

```
const todos = [  
  { id: 1, text: "Install Node", done: true },  
  { id: 2, text: "Learn JS",      done: false },  
];
```

Write functions that **return a new array** without changing the original:

1. `addTodo(todos, text)` – adds a new todo (unique `id`, `done: false`)
2. `toggleTodo(todos, id)` – flips `done` of the todo with that `id`
3. `removeTodo(todos, id)` – removes the todo

At the end, print `todos` – it must be **unchanged**.

Hint: `[...arr, x], .map(t => t.id === id ? { ...t, done: !t.done } : t), .filter()`

Mini-project: Formatting JSON data

Task:

Create a script `formatUsers.mjs` that:

1. Fetches users from `https://jsonplaceholder.typicode.com/users` (with `async/await`)
2. Extracts the name, email and city (with destructuring)
3. Sorts them alphabetically by name
4. Prints them nicely formatted:

```
👤 Leanne Graham  
✉ Sincere@april.biz  
🏠 Gwenborough  
-----
```

Bonus: Put the formatting in a separate module (`format.mjs`) and `import` it.

Resources

- Marijn Haverbeke, Eloquent JavaScript, 4th edition (2024), CC BY-NC
<https://eloquentjavascript.net>
- JavaScript Guide by Mozilla MDN
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- JavaScript.info
<https://javascript.info>
- React — JavaScript you need for React
<https://react.dev/learn/javascript-in-jsx-with-curly-braces>
- JSFiddle test bed
<https://jsfiddle.net>

